

What Is Work

The Law of Information Conservation and the
AI Productivity Paradox in High-Context Knowledge Work

Alexander Vityaz

Corezoid Inc., Dnipro, Ukraine

alexander.vityaz@gmail.com

corezoid.com · simulator.company · NotebookLM

April 2026

1. The Promise That Does Not Add Up

The artificial intelligence industry markets a single metric: time to first result. A prompt—thirteen seconds—a working prototype. The implied conclusion is that programming has accelerated by an order of magnitude.

I write documents and code with AI every day. My experience contradicts this promise. The prototype appears quickly. But to bring it to a working state, I spend hours refining prompts, verifying outputs, rolling back errors, manually writing or editing text, and iterating again. The total time required to complete a task of a given quality has not decreased. What has changed is something else: previously I worked with an engineer; now I work alone.

This forces a question that is usually not asked: what is work from an informational point of view, and does AI actually change its volume?

2. Three Laws

I call these laws in the tradition of Ashby and Shannon—not as claims about nature, but as **invariants that hold by construction**. Their value lies not in falsifiability, but in redirecting attention. The first law specifies an invariant (what is conserved), the second a constraint (what constrains), and the third a structure (what factorizes). Everything else in the essay follows from these three.

The Law of Information Conservation

Every act of creation reduces to a single operation: input information is transformed into an output artifact. The quality of the artifact Q is determined by the volume of information taken into account: $Q = f(I_{processed})$. A programmer who has accounted for edge cases, security requirements, and

codebase conventions will produce higher-quality code than one who has accounted only for the main scenario. Quality is a measure of information accounted for.

For an artifact of a given quality Q , the volume of required information I_Q is fixed. It is determined by the task, not by the tool:

$$I_Q = \text{const} \quad \text{for fixed } Q \quad (1)$$

This is the **law of information conservation**—a consequence of the law of requisite variety (Ashby, 1956): the regulator must possess variety at least equal to that of the system being regulated, and every good regulator must be a model of that system (Conant & Ashby, 1970). One cannot create an artifact of quality Q by processing less information than Q requires. The law operates in the direction one usually ignores: the industry optimizes T (generation speed), whereas the fundamental constraint is I_Q . No matter how fast the tool works, I_Q does not decrease; only the identity of the processor changes.

Time T is not an invariant: it is determined by processing speed, $T = I_Q/V_{\text{processing}}$, and can decrease with better tools. But I_Q is invariant. And because $V_{\text{processing}}$ is limited by human cognitive throughput (see below), in practice the initiator's time $T_{\text{initiator}}$ does not decrease either.

Units and operational proxies for I_Q . The dimension of I_Q is bits in Shannon's information-theoretic sense: the logarithm of the number of distinguishable variants of the artifact at fixed Q . Direct measurement of this quantity is impossible in open domains (a codebase, a text, a business decision), so I_Q serves as an analytical invariant rather than a metric magnitude. For operational use, three proxies are available, each measuring a partial trace of I_Q : (1) the number of **independent decisions** made in the course of creating the artifact (each contributing $\log_2$ of the number of alternatives considered); (2) the **minimum description length** sufficient for a competent team to reproduce the artifact without additional clarification; and (3) the number of **clarifying questions** that an independent executor would need to ask the sender in order to complete the task. None of these proxies is equivalent to I_Q , but all three correlate with it and can be measured in controlled experiments. For the purposes of this essay, it is enough that I_Q is uniquely defined at fixed Q ; the concrete numerical value depends on the domain and is not required for the model's consequences.

Corollary 1. Moving from N to $N - 1$ participants does not reduce I_Q . The volume of information remains the same; only the distribution of the burden changes.

Corollary 2. If $N \rightarrow 1$ and the sole participant is the initiator, the entire processing of I_Q falls on that person: $T_{\text{initiator}} = I_Q/V_{\text{cognitive}}^{\text{initiator}}$.

Corollary 3. The only way to reduce $T_{\text{initiator}}$ at fixed Q is to reduce I_Q through compression.

The Law of the Bottleneck

The speed of any channel equals the speed of its slowest participant (Shannon, 1948; Wiener, 1948):

$$V_{\text{transfer}} = \min(V_{\text{out}}, V_{\text{in}}) \quad (2)$$

Human $\rightarrow$ **AI**: $V_{out} \approx 40$ wpm (typing), $V_{in}^{AI} \approx \infty$. The bottleneck is the human. **AI** $\rightarrow$ **Human**: $V_{out}^{AI} \approx 800$ words/s (~ 1000 tokens/s), $V_{in} \approx 4$ words/s (reading code with comprehension). Again, the human. In both directions, the bottleneck is the human. AI generates roughly 200 times faster than a human can read. This gap does not create acceleration; it creates a queue of unread material.

The reason is not the speed of the eyes, but the capacity of working memory: 7 ± 2 items (Miller, 1956), refined to 4 ± 1 for unchunked elements (Cowan, 2001). A typical agent response is 200 lines of code. To verify it, one must simultaneously hold in mind the data structure, the current function, the calling context, the loop invariant, boundary conditions, variable names, and the expected result. That amounts to 7–12 **chunks** (units of cognitive processing that package related elements into a single bundle). Working memory overflows—this is the classical effect of cognitive overload (Sweller, 1988).

$$V_{in}^{effective} = \frac{M}{\tau_{chunk}}, \quad M = 7 \pm 2, \quad \tau_{chunk} \approx 2\text{--}5 \text{ s for code} \quad (3)$$

The ceiling is $V_{in} \approx 1.5\text{--}3.5$ chunks/s—a neurophysiological constant. AI operates on roughly 100,000 chunks in parallel (the context window); the human on roughly 7. But the decision about correctness is made by the human. The limit of the human+AI system is therefore the limit of the human:

$$T_{initiator} \geq \frac{I_{total}^{read} + I_{total}^{write}}{V_{cognitive}^{human}} \quad (4)$$

The Law of Factorization

The minimal good regulator factorizes into a model of the system and a noise filter (Vityaz, 2026a):

$$\text{Regulator}_{min} = \text{Model} \times \text{Filter} \quad (5)$$

I state explicitly the formulation on which the essay relies.

Theorem 1 (Vityaz–Ashby, 2026). *Every good regulator of a system S must contain an effective noise suppressor that separates essential disturbances from inessential ones. Consequence: the minimal good regulator factorizes into a model of the system and a noise filter.*

The proof rests on three pillars: (1) the Conant–Ashby theorem (the regulator must be a model of the system); (2) Ashby’s law of requisite variety ($|R| \geq |D|/|E_{admissible}|$, where $|R|$ is the variety of the regulator, $|D|$ is the variety of disturbances, and $|E_{admissible}|$ is the variety of admissible outcomes); and (3) the decomposition of disturbances $d = \sigma(d) \oplus \eta(d)$, where σ is the signal component (essential disturbances) and η the noise component (inessential ones).

Without filtering, the regulator’s variety grows multiplicatively: $|R| \geq |\sigma(D)| \cdot |\eta(D)|$ —an exponential explosion of complexity. With a filter $F : D \rightarrow \sigma(D)$ that suppresses η , one obtains $|R_{effective}| \geq |\sigma(D)| \ll |D|$. By contradiction, a regulator without noise suppression either contains redundant states (hence is not minimal) or generates false control actions. In either case, it is not “good.”

A team working on a task is a regulator. The task is the regulated system. I_Q includes not only useful signal but also the unavoidable cost of filtering noise. The question that determines everything

else is this: **who performs the “Model” component, and who performs the “Filter” component?**

This factorization is the essay’s generative principle. Everything that follows—the paradox, context, threshold, the pair, the paths—follows from one question: how are Model and Filter distributed across participants?

Determinism of the Filter

This section introduces the qualitative requirement on which §§7 and 9 rely. A strict multiplicative version that separates type I and type II errors is the subject of a separate mathematical work; for the purposes of this essay, the following indicative claim is sufficient: an error-prone filter violates the minimality of the regulator.

An important clarification following from the proof of the theorem (Vityaz, 2026a) is that the filter must be **deterministic**. This is not an implementation convenience but a necessary condition of minimality.

Consider a filter F_ε that errs in a fraction ε of cases: with probability $1 - \varepsilon$ it passes signal, and with probability ε it passes noise. A regulator built on such a filter must compensate for these errors—that is, it must contain redundant variety of states for handling false positives. Formally:

$$|R_\varepsilon| \geq |\sigma(D)| + \varepsilon \cdot |\eta(D)| \quad (6)$$

As $\varepsilon \rightarrow 0$, we obtain the minimal regulator. For $\varepsilon > 0$, the regulator is **strictly larger than the minimal one**—and therefore is not good in the sense of the Conant–Ashby theorem. A statistical filter (that is, an error-prone one) yields a regulator that is either non-minimal or prone to false control actions.

Note on rigor. The additive form of the inequality is a first approximation. A full treatment requires separating two types of errors: type I (signal discarded as noise, which requires redundant routes in the regulator) and type II (noise passed as signal, which requires verification states). These two types contribute multiplicatively rather than additively to $|R|$. For the purposes of the present essay, the first approximation is sufficient; formalizing the multiplicative version is a task for separate mathematical work.

This distinction becomes critical in §§7 and 9 when analyzing possible ways of constructing an AI filter: it determines which architectures are compatible with the theorem and which are not.

Key Symbols: A Brief Glossary

For the reader’s convenience, here are the seven main quantities that appear in §§3–§9 before their formal introduction:

Symbol	Meaning	Defined in
I_Q	Volume of information required for an artifact of quality Q	§2 (above)
C_{shared}	Common codebook of sender and receiver (Shannon, 1948)	§4
α	Compression coefficient: $\alpha = I_{explicit}/I_{total}$ (low = good compression)	§4
β	Share of the initiator's time when working with AI (high = the initiator is constantly in the loop)	§5
γ	Share of the initiator's time under delegation (low = good delegation)	§5
ρ	Information-retention coefficient on upward paths	§7
L	Share of irrecoverable losses under re-encoding	§7

The formulas for α , β , γ , ρ , and L will be introduced in the course of the argument. Here only their role in the model is given.

3. The AI Productivity Paradox

Delegation as the Multiplexing of Time

Delegation is the multiplexing of a scarce resource (the initiator's time) through a lossy communication channel (Brooks, 1975). I sketch an idea on a whiteboard. Five minutes. The engineer looks at it, asks clarifying questions, leaves, and implements it over two days.

$$T = T_{initiator} + T_{engineer}, \quad T_{initiator} = 5 \text{ min}, \quad T_{engineer} = 2 \text{ days} \quad (7)$$

The initiator's share is $T_{initiator}/T \approx 0.5\%$. The rest of the time I am free. Throughput is $B_{initiator} = T_{available}/T_{initiator \text{ per project}}$. At 20 minutes per day per project, that is up to 24 projects in parallel in theory, and practically 5–10 once context switching is taken into account.

Delegation works because the engineer carries both components of the factorization. The engineer is the **model** of the codebase (knows the conventions, the history of decisions, the dependencies) and the **filter** (verifies their own code, filters errors before sending anything back to the initiator). The initiator performs neither component, only setting the direction.

The AI Agent as Demultiplexer

Now consider the same task with AI. I describe the data structure and specify the constraints. The agent generates code. The code contains an error—I describe it. The agent fixes it but introduces a regression. I clarify the context. After 47 iterations, the code works.

$$T = T_{initiator} + T_{agent}, \quad T_{agent} \rightarrow 0 \implies T_{initiator} \rightarrow T \quad (8)$$

All informational work—verification, clarification, transmission of context, iterative correction—

falls on the initiator. Writing code is only a small part of that work. Why does T not decrease, even though AI generates hundreds of times faster? Because the bottleneck is the initiator’s cognitive throughput (§2), and that does not depend on model speed.

In the language of factorization, AI is a **generator without a filter**. It has no model of the code-base ($C_{shared} \approx 0$), no model of the initiator as the receiver, and no judgment. It outputs 200 lines, of which 10 matter—but the initiator determines which 10. The factorization collapses: the human is forced to be both model and filter at once. This is what appears empirically as $\beta \approx 0.9$: the human does almost everything.

The AI agent removes the need for other participants, but it does not remove the need for informational work. All the work saved on coordination returns to the initiator:

$$T_{initiator}^{AI} \approx T \gg T_{initiator}^{delegation} = T - \sum T_{delegate_k} \quad (9)$$

Parameter	Delegation to engineer	Direct work with AI
I_Q	100%	100%
$T_{initiator}$	$\approx 5\text{--}20\%$	$\approx 100\%$
Factorization	Engineer = model + filter	Human = model + filter; AI = generator
Bottleneck	Engineer (days, autonomous)	Initiator (hours, continuous)
Parallelism ($B_{initiator}$)	5–10 projects	1 project

Analogy: Level 2 Autopilot

The paradox becomes clearer through an analogy with autonomous driving. The SAE (Society of Automotive Engineers) classification defines six levels of automation—from Level 0 (fully manual control) to Level 5 (fully autonomous). The current state of AI agents in knowledge work corresponds to **Level 2: partial automation**.

At Level 2, the car controls steering and pedals, but the driver **must keep their hands on the wheel and eyes on the road at every moment**. The car generates motion; the driver remains responsible for monitoring, correction, and takeover. This is exactly $\beta \approx 0.9$: AI generates code, and the human verifies every line.

A counterintuitive fact from the automotive industry is that **driver fatigue at Level 2 is higher than at Level 0**. Monitoring autopilot is cognitively harder than driving manually, because the brain must simultaneously build a model of the road and check the autopilot’s decisions. Two streams instead of one. Lisanne Bainbridge formalized this effect in the classic paper “Ironies of Automation” (Bainbridge, 1983): the more a system is automated, the more difficult the remaining operator’s task becomes, because the operator is left precisely with those functions that could not be automated (judgment-intensive decisions, anomaly handling, supervision). The AI productivity paradox is a special case of this irony: the tool “helps,” but cognitive load increases.

The “mis-calibrated trust” documented by Dell’Acqua et al. (2023) is the knowledge-work analogue of Tesla crashes at Level 2: the driver trusts the autopilot more than warranted, removes their hands from the wheel, and the result is worse than without autopilot. Tasks “beyond the frontier” are

roads the autopilot did not see in the training set.

The hidden cost axis: the regime of suspicion. Delegation to an engineer operates on *trust*: “you will handle it and show the result.” Trust is a **form of context compression**: the receiver does not need to re-verify what the sender has already accepted. Working with AI operates in the opposite regime—*constant suspicion*: “what has it gotten wrong this time?” Suspicion is not merely unpleasant; it is **cognitively more expensive** even than the Zeigarnik loop. If Zeigarnik is passive retention of an unfinished task, suspicion is active error search, System 2 in a forensic-analysis mode. In the terms of §2, suspicion is a **refusal of compression**. A human who does not trust the output of the tool is forced to unfold the full verification of every bit—in other words, to process the same I_Q from which the tool was supposed to relieve them. Dell’Acqua’s mis-calibrated trust describes both poles: too much trust destroys safety (Tesla crashes); too little destroys productivity (effectively manual work with an expensive tool in the loop).

In factorization terms:

SAE Level	Model (where to)	Filter (how safely)	Analogue in this essay
0: No automation	Driver	Driver	Human without AI
2: Partial	Driver	Driver (monitoring) + car (execution)	Current state: Human = model + filter; AI = generator
3: Conditional	Driver	Car (with right of takeover)	AI filters, human takes over under anomaly
4: High	Driver (strategy)	Car	Mature pair: AI with C_{shared} , human sets direction
5: Full	No driver	Car	Not foreseeable for knowledge work

Level 5 is not visible on the horizon under the current architecture of context transmission, for the same reasons as in arbitrary driving conditions: the five layers of context ($C_{implicit}$, $C_{ambient}$, C_{social} , $C_{nonverbal}$, lived experience) cannot be transmitted to an autopilot through a text channel. The car does not possess the model of meeting urgency available to the driver. AI does not know that a deadline is a promise rather than a date. Whether Level 5 is reachable under a fundamentally different architecture remains an open question. The pragmatic task is the transition from Level 2 to Level 4: to give AI sufficiently deep C_{shared} that it becomes a filter rather than a generator of noise.

AI has truly changed three things. **The entry threshold:** tasks that previously required hiring a specialist are now accessible to a single person. This is not acceleration; it is an expansion of the set of people able to perform the task. **Communication losses:** $L_{comm} \rightarrow 0$, but this is offset by the need to describe context explicitly. **Latency:** instead of a three-day wait with parallel work, one gets six hours of continuous chat with no context switching.

Empirical Confirmation

The model predicts that AI accelerates tasks below the threshold (I_Q is small, context unnecessary) but slows tasks above the threshold (I_Q is large, context critical). This is exactly what is observed experimentally.

Peng et al. (2023) showed that GitHub Copilot accelerates the implementation of an HTTP server by 55.8%—a task with zero codebase context, one file, one language. Dell’Acqua et al. (2023) identified a “jagged technological frontier”: for tasks inside the frontier, AI raises productivity; for tasks beyond it, productivity falls by 19 percentage points. Tasks beyond the frontier involve the integration of quantitative and qualitative data, organizational context, and judgment—precisely the domains in which C_{shared} is critical and $\alpha^{human \rightarrow AI} \approx 1$.

In the terms of the present model, Dell’Acqua’s “frontier” is the threshold W^* . Peng et al. (2024), using 4,900+ developers at Microsoft and Accenture, confirmed heterogeneity of the effect: the largest gains go to less experienced developers (simpler tasks, less need for context).

Historical precedent—the Jevons paradox. In 1865, William Stanley Jevons observed what would later bear his name: increases in the efficiency of steam engines (they burned coal more economically per unit of work) did not reduce, but rather **increased**, total British coal consumption. Cheap coal opened new uses—steamships, railroads, industrial-scale production—that had previously been economically nonviable. Efficiency created demand that absorbed and exceeded the savings. LLMs are the steam engine of knowledge work: making code generation cheaper opens classes of tasks that were previously not done at all (rapid prototypes, experiments, automation of the long tail of scripts). The total volume of generated code grows faster than the unit cost falls. Organizational T_{total} does not decline and may even rise—the exact same mechanism as in 1865.

Limitation: the model was developed with knowledge of these results, so the explanation is retrospective—an illustration rather than independent confirmation. Stronger evidence would come from **prospective predictions** that can be falsified:

- The threshold W^* should decline as team tenure increases (growth of $C_{shared} \rightarrow$ decline of $T_{overhead} \rightarrow$ shrinkage of the AI zone). This is testable by comparing W^* across teams with different tenure.
- AI pair programming should yield larger gains on refactoring tasks (low C_{shared} with context) than on feature-integration tasks (high C_{shared}). This is testable within an RCT (randomized controlled trial) of the type used by Peng et al.
- Persistent AI memory (growth of $C_{shared}^{human \rightarrow AI}$ across sessions) should lower β and expand the AI zone. This is testable by comparing β with and without persistent memory.

Caveat: External Validators

The picture “AI = generator without filter” is accurate for the bare pair Human $\leftrightarrow$ LLM (large language model—the basis of modern AI agents). In practice, however, a large share of filtering in programming is outsourced to the executable loop: the compiler rejects syntactically invalid code,

types catch signature errors, the linter catches stylistic violations, tests catch functional errors, and CI catches integration errors. This is an **existing deterministic filter** that operates independently of the LLM.

Where that loop is rich (typed languages, well-tested codebases), β is lower in practice: the human is not obliged to verify every line of AI output—part of the verification will be done by the compiler in milliseconds. Where the loop does not exist (a chatbot prompt, an untyped Python script, a black-box API), β approaches the theoretical boundary of ~ 0.9 .

This is a special case of AImatics (§9): an executable verification loop is a deterministic filter. The hybrid architecture generalizes this pattern to non-code domains where such a loop does not exist by default.

Descriptive vs. Normative

From this point on, I will describe AI in two registers. **Descriptively** (§3, §4): AI currently functions as a generator without a filter—this is the empirical state of affairs in 2026. **Normatively** (§7, §9): in a pair, AI **ought to be** the Filter component—which follows from the theorem of factorization. The gap between these two registers is the subject of the essay: how to move from one to the other.

4. Context: Why Humans Compress and AI Does Not

Corollary 3 from §2 points to the only path to real acceleration: reducing I_Q through compression:

$$I'_Q = I_Q \cdot \alpha, \quad \alpha < 1 \quad (10)$$

Compression is possible when sender and receiver share a **common codebook** C_{shared} (Shannon, 1948):

$$\alpha = \frac{I_{explicit}}{I_{total}} = 1 - \frac{C_{shared}}{I_{total}} \quad (11)$$

The Five Layers of the Codebook

C_{shared} between humans is not a flat array of facts. It is a layered structure in which verbalizable knowledge is only the tip of the iceberg (Polanyi, 1966; Nonaka & Takeuchi, 1995):

$$C_{shared} = C_{explicit} + C_{implicit} + C_{ambient} + C_{social} + C_{nonverbal} \quad (12)$$

$C_{explicit}$ is explicit knowledge: documentation, code, specifications, README files. It is the only layer at least partially accessible to AI through the context window.

$C_{implicit}$ is tacit knowledge (Polanyi, 1966): “this is how we do things here.” It is written down nowhere and is transmitted through practice: we do not use an ORM in this project; Friday deployment is taboo; if a PR exceeds 300 lines, it will not be accepted.

$C_{ambient}$ is environmental context. Both people attended the stand-up; both heard that the back-end team is delaying the API; both saw the error dashboard climbing. This context is not transmitted; it is shared through co-presence in the same environment. At the whiteboard one can say, “well, you

saw it this morning,” and the engineer understands.

C_{social} is a model of one another. The engineer knows that when I say “quickly,” I mean “by Thursday,” not “optimal in Big-O terms.” They know my blind spots. They know I do not need REST explained to me but do need gRPC explained. I know that I can trust them with architecture but need to check the UI.

$C_{nonverbal}$ is the nonverbal channel: the pause before the word “async” at the whiteboard; the gesture meaning “this will be hard here”; the facial expression meaning “this is critical”; the tone of voice meaning “I am not sure about this decision.” These signals modulate verbal content and carry information that text does not convey.

Layer	Character	Accessible to AI?
$C_{explicit}$	Formalized, transmissible through text	Partially (context window)
$C_{implicit}$	Non-formalized, transmitted through practice	No
$C_{ambient}$	Shared through co-presence in an environment	No
C_{social}	Mutual models of participants	No
$C_{nonverbal}$	Modulation of the verbal channel (tone, gesture)	No

The exact proportions depend on the domain, team maturity, and task type—and cannot be measured precisely. But Nonaka & Takeuchi (1995) showed that in Japanese corporations the dominant share of organizational knowledge is tacit, while explicit (formalized) knowledge is only a small fraction. Polanyi (1966) stated the point more strongly: “we can know more than we can tell.” The invariant that does not depend on exact proportions is this: **AI sees only the top layer ($C_{explicit}$); the remaining four are in principle inaccessible through a text channel.** Even if $C_{explicit}$ is 20% rather than 10%, AI still does not see 80%. The order-of-magnitude difference remains.

Depth: Lived Experience vs. Textual Snapshot

The difference between human context and AI context is not quantitative but categorical.

When an engineer says, “I would not make this synchronous here,” behind that statement stands a night spent in production with a deadlock three years ago. That bit of context is not fully verbalizable, yet it is exactly what determines judgment. It is written nowhere—it has been lived. Painful experience creates a prioritization that cannot be transmitted by text. An attempt to explicate “why not sync” will take a page and still fail to communicate the feeling of “never again.”

$$C_{human} = f(\text{experience, time, body, emotions, society}) \quad (13)$$

$$C_{AI} = f(\text{tokens in window}) \quad (14)$$

Axis	Human	AI
Breadth (RAM)	~7 chunks (Miller)	~100K+ tokens
Depth	Decades (career, life)	One session
Structure	Hierarchical (the important sits on top)	Flat (all tokens are equal)
Modality	Multidimensional (vision, body, emotion, sound)	One-dimensional (text)
Valence	Emotional (pain = “do not do this”)	Zero (no pain → no priority)

AI wins on breadth by 10^4 . Humans win on the other four axes—and those advantages are not compensable by breadth. The table describes **what each has**. More important, however, is **what each type of context enables one to do** and how depth amplifies each function.

Five Functions of Context = Five Noise Filters

Function	Human	AI
Prediction	“Vasya will oppose this approach,” “the CEO will ask about deadlines first” — context as a predictor of people and systems	Answers to queries from the text in the window — context as a reference source
Intuition through pain	“NO, not sync” — in 0.1 s, without deliberation. A night in production → instantaneous System 1 (Kahneman, 2011). Scars create prioritization	Every question is System 2: reasoning from text. No scars → no intuition
Networked context	Distributed across a network of people: I know what the engineer knows; the tech lead knows whom to trust. Metacontext = context about the contexts of others	A point object: it does not know what others know and does not model their contexts
Real time	The CTO frowned at the word “microservices” → the next decision already takes that into account	Snapshot: fixed at the moment of the prompt. Between prompts, the world has changed
Filtering	A 200-line PR → “line 47 looks suspicious” in 30 s. Learned attention (Wiener, 1948; Vityaz, 2026a)	All 200 lines look equivalent. Attention is mechanistic

Each function is a mechanism of **noise suppression** that uses a model of the system (C_{shared}) as a filter. Each is strengthened by depth of context: prediction becomes more accurate after three years of working with Vasya; intuition appears after a night in production; networked context thickens recursively (after two years I know not only what the engineer knows, but also what the engineer knows about what I know); filtering grows more precise with every PR read.

Depth is not simply “more data.” It is a **qualitative change**: as experience accumulates, context moves from reference material ($C_{explicit}$) to intuition ($C_{implicit}$) and then to instant pattern matching ($C_{nonverbal}$). AI context does not traverse this path—it always remains on the first level, no matter how wide the window becomes.

In factorization terms (§2), the five functions are five components of the **noise filter**, and all of them require a **model** (C_{shared}) in order to operate. AI has no model; therefore none of the filters works (Vityaz, 2026a). The result is straightforward: no prediction $\rightarrow$ expected reactions must be described. No intuition $\rightarrow$ every “no” must be justified. No networked context $\rightarrow$ one must retell what others know. No real-time awareness $\rightarrow$ the prompt must be updated manually. No filtering $\rightarrow$ the entire output must be read.

$\alpha^{human \rightarrow human} \ll \alpha^{human \rightarrow AI}$ not by a factor of 2 or 5, but by an order of magnitude: five filters operate multiplicatively, each is amplified by depth, and the difference is both one of type and one of depth.

Upper bound: $C_{shared}^{human \rightarrow AI} \leq C_{explicit} \ll C_{shared}^{human \rightarrow human}$. The exact coefficient depends on the domain, but the order-of-magnitude difference is stable: AI sees one layer out of five. Even an infinite window does not compensate for this: text does not carry body, emotion, environment, or history. C_{shared} is an accumulated investment ($C_{shared}(0) \approx 0$, $C_{shared}(t) \rightarrow C_{max}$). AI context is rented without amortization (it resets between sessions in the baseline configuration).

Epistemological Asymmetry: We Do Not Know What We Know

The five-layer model has a consequence more troubling than arithmetic: **tacit knowledge is, by definition, not inventoryable**. When an engineer writes a prompt for AI, the engineer does not know what exactly they forgot to transmit. They do not know which of their own $C_{implicit}$, $C_{ambient}$, C_{social} , and $C_{nonverbal}$ assumptions remained outside the text. They discover this only post hoc—through a bug that surfaces in production, through a solution that turns out to be inapplicable, or, in the worst case, through a catastrophe that no one notices in time because the system “worked correctly” within the assumptions it was given.

This is what fundamentally distinguishes work with AI from delegation to a human. A human receiver **also possesses tacit knowledge**—and when the specification is incomplete, that person reconstructs from their own C_{shared} : “Friday deployment is taboo, so this is not an urgent hotfix, so we need to discuss it with the architect.” A human **fills the sender’s omissions** with their own knowledge. AI does not do this. AI calmly operates within the reduced world it has been given, without noticing that the world has been reduced.

The paradox is that **the human receiver has more information about what was not transmitted than the sender does**. The engineer does not know that they forgot to mention multicurrency support—but a senior colleague, reading the task, immediately asks: “Have you thought about JPY?” AI does not ask. AI proposes.

In factorization terms (§2), this is the most insidious case of a type II error: not merely noise passed as signal, but **the absence of signal itself going undetected**. A filter cannot suppress what it does not know about.

Three Channels

Human $\rightarrow$ Human. Three rectangles on the whiteboard, “async.” Five minutes. The engineer leaves—and two days later returns with 2,000 lines of code. The drawing: ~ 50 bits. The result: $\sim 200,000$ characters. Amplification by four orders of magnitude. The engineer filled in from all five

layers of the codebook. In a mature team: $\alpha \ll 1$.

Human $\rightarrow$ AI. $C_{shared} \leq C_{explicit}$: four of the five layers are not transmissible through text. $\alpha \approx 1$. Every omitted bit returns as an error: $\Delta I_{omitted} \rightarrow \Delta n_{iterations} \rightarrow \Delta T_{initiator}$.

AI $\rightarrow$ Human. $C_{shared} \approx 0$ —the agent has no model of the receiver and therefore does not filter the output. Reading efficiency $\eta = I_{new}/I_{response} \ll 1$. An experienced engineer will say, “I did it the way we did last time, just with a different endpoint”—three seconds, $\eta \approx 1$, pure signal. The agent emits the full uncompressed, unfiltered response.

5. Threshold: When AI, When Human

A **task** is an atomic unit of work: one executor, one input, one output, at most one day. Anything larger is a **project**: $W = \sum W_{task_k}$, $N \geq 2$. A task longer than one day contains hidden dependencies—decomposition is mandatory.

Status of the Numerical Values

All numerical values in this section are **calculated, not measured**. They are obtained by substituting expert estimates of parameters (β , γ , $T_{overhead}$) into the derived formulas. The specific figures are illustrative, not experimental results. The structure of the model is more robust than the numbers: changing the parameters by 30% shifts the thresholds proportionally—the qualitative picture (AI is optimal for microtasks, delegation for larger ones) survives under any reasonable values. The reader’s task is to substitute their own parameters for their own team and obtain their own thresholds. The relevant formulas follow below.

The Threshold from the Compression Model

The compression model of §4 predicts the direction and order of magnitude of two key coefficients: $\alpha^{human \rightarrow AI}$ is high (no compression) $\rightarrow \beta$ (the initiator’s share of time with AI) should be high—the initiator remains continuously in the loop. $\alpha^{human \rightarrow human}$ is low (good compression) $\rightarrow \gamma$ (the initiator’s share under delegation) should be low—the initiator hands over a small share. This is not a mathematical identity: β measures a share of time, α a share of information, so their units differ. But the compression model correctly predicts that $\beta \gg \gamma$ and estimates the order: $\beta \sim 0.9$, $\gamma \sim 0.1$.

With AI: $T_{initiator}^{AI} = \beta \cdot W$ (prompts + verification + iterations; AI absorbs $1 - \beta \approx 10\%$ —pure generation). With delegation: $T_{initiator}^{del} = T_{overhead} + \gamma \cdot W$ (task framing + latency + review; $T_{overhead} \approx 30 \text{ min} = 5 \text{ min framing} + 15 \text{ min waiting} + 10 \text{ min review}$).

$T_{overhead}$ depends on C_{shared} with the delegate: with an experienced engineer, 5 minutes at the whiteboard; with a novice, 45 minutes of explanation. The threshold W^* itself therefore depends on team maturity.

$$W^* = \frac{T_{overhead}}{\beta - \gamma} = \frac{30}{0.8} = 37.5 \text{ min} \quad (15)$$

Hidden Multiplier: $\beta_{cognitive}$ vs. β_{active}

Defining β as the “share of the initiator’s time” requires refinement. The work cycle with AI looks like a sequence: [15 s prompt] $\rightarrow$ [30–90 s waiting] $\rightarrow$ [2 min verification] $\rightarrow$ [15 s next prompt] $\rightarrow$ [60 s waiting]... The waiting intervals are too short to switch to another meaningful task and too long for comfortable waiting. This gives rise to a distinction:

- β_{active} — the share of time spent in **active actions** (typing a prompt, verifying a response)
- $\beta_{cognitive}$ — the share of time during which **the mind is occupied by the task** (active actions + waiting)

The mechanism linking them is the **Zeigarnik effect** (Zeigarnik, 1927): an unfinished task retains working-memory resources roughly twice as strongly as a finished one, creating residual activation in the prefrontal cortex. Every open prompt is a Zeigarnik loop: we wait for the answer, prepare to verify it, and cannot mark the task as “done.” This loop actively occupies Miller’s working-memory slots even in the absence of overt action.

The effect connects directly to §2: if $M \approx 7$ slots is Miller’s working-memory capacity, then n simultaneously open prompts shrink the available window from M to $M - n$. Add to this the interruption effect (Mark, 2004): ~ 23 minutes to recover focus after each attention switch. Altogether:

$$\beta_{cognitive} \geq \beta_{active} + \tau_{wait} \cdot \mu \quad (16)$$

where τ_{wait} is the share of waiting time and μ the Zeigarnik resource-retention coefficient.

Calibration of μ . The value of μ is not a constant but a function of waiting conditions. The anchor point is the original Zeigarnik (1927): unfinished tasks are retained in memory approximately twice as strongly as completed ones, a result replicated multiple times (Mäntylä & Sgaramella, 1997; Koshino et al., 2014—frontoparietal sustained activation for open tasks). This is a **retention** coefficient, not a direct coefficient of working-memory occupancy in real time; the relation between the two is mediated.

In AI-waiting practice, plausible ranges are:

- $\mu \approx 0.5\text{--}0.7$ — when a predictable ETA exists (a progress bar with time estimate) and an interruptible task exists that fits the pause length
- $\mu \approx 0.7\text{--}0.9$ — the typical case: waiting without ETA, no suitable alternative task
- $\mu \rightarrow 1.0$ — when the verification stage is critical (production code, legal text) and the brain keeps “prepare to verify” in an active working state

For everyday use in high-context engineering work, $\mu \in [0.7; 0.9]$ is a rough first approximation, to be refined by empirical measurements. The exact value matters less than the qualitative fact: $\mu > 0$, and substantially so—waiting is not free from the standpoint of working memory.

This explains a clinical phenomenon: engineers working with AI may work fewer hours (β_{active} falls), yet feel more exhausted by the end of the day ($\beta_{cognitive}$ rises). Generation speed has increased; **the density of cognitive load has increased as well**. The total effect on productivity requires measuring both quantities, not only clocked time.

Practically, this means that low-latency tools (response time < 15 s) are qualitatively better than high-latency tools (response time > 60 s) even at equal accuracy—because short response times do not have enough time to create a stable Zeigarnik loop.

The Main Damage: Managerial Parallelism

Before introducing opportunity costs arithmetically, it is worth naming what is actually lost. $B_{initiator} = T_{available}/T_{initiator\ on\ project}$ is not an abstract throughput coefficient. It is **the defining property of the manager's role**.

When a manager delegates to an engineer, the manager spends 5–10 minutes at the whiteboard per project and is then free for hours or days. Throughput is $B_{initiator} \approx 5\text{--}10$ projects in parallel. This is not a “convenience”; it is what **makes the person a manager**. A manager is not someone who works on one task better than others. A manager is someone who simultaneously keeps many tasks in focus, allocating limited attention across them.

When a manager works directly with AI, $T_{initiator} \rightarrow T$ (§3): one task consumes all of the manager's working time. $B_{initiator} \rightarrow 1$. **This is a qualitative change in role disguised as a quantitative change in productivity**. On the velocity dashboard, the manager “completed a task in a day.” In reality, the manager lost four other projects that were supposed to be run in parallel.

This is precisely why the manager's thresholds W^{**} shift into microtasks ($\sim 5\text{--}10$ min, not tens of minutes). Not because the manager has higher quality requirements, but because **every minute spent outside a microtask directly destroys the parallelism on which the role rests**. A manager who systematically works with AI on hour-long tasks ceases to be a manager and becomes a senior individual contributor with a wider zone of responsibility and the same daily time budget.

Formal consequence: accumulation of management debt. This loss is not abstract—it is measurable within the formal framework of management debt (Vityaz, 2026b). A manager is the Decision Owner for many open managerial questions at once. Each directive (a request for a decision) addressed to the manager but not closed by an assertive (the fixation of a decision) within the required time generates **omission debt** on the manager's Actor Account—a formally tracked object with a date of origin, a Decision Owner, and a link to direct loss upon materialization. As long as the manager runs 5–10 projects in parallel with $T_{initiator} \approx 0.5\%$, the flow of directives is processed; omission debts are closed faster than they accumulate. When $B_{initiator} \rightarrow 1$, this equilibrium breaks: decisions on 4 out of 5 projects **are not taken**, not because they are taken badly, but because the manager is **physically absent from the corresponding managerial control loops**—fully absorbed in a single debugging session with AI. Other classes atrophy in parallel: delegation debt (tasks are not delegated because the manager does them personally), priority debt (priorities are not re-evaluated), and management observability debt (visibility over the remaining projects disappears). The velocity of one task rises, while the balance-sheet accounting of the role deteriorates. In sufficiently measurable organizations, this can be shown through Actor Account dynamics: weeks of intensive work with AI coincide with growth in the balance of unmade decisions.

With Opportunity Costs

The single threshold W^* ignores the fact that the initiator's time is not a free resource. When the initiator runs P projects in parallel, every minute spent on AI work in project k is a minute not spent on projects $k' \neq k$. Let v denote the **value of alternative time** (the share of $T_{initiator}^{AI}$ that could have been redirected to another project had this one been delegated). At $v = 1$, time is fully interchangeable across projects; at $v = 0$, there is no alternative (one project = one unit of attention).

The full cost of direct work with AI is therefore not only $\beta \cdot W$ in the current project, but also the loss of the opportunity to work on $P - 1$ other projects. A linear approximation (each of the $P - 1$ projects loses the same share of attention) gives:

$$T_{initiator}^{AI, effective} = \beta \cdot W \cdot [1 + (P - 1) \cdot v] \quad (17)$$

Linearity is a first approximation. For small P it is justified: switching between projects is cheap, and the loss is proportional to the count. At larger P , nonlinear effects emerge (Brooks, 1975: quadratic communication costs), but for $P \leq 10$ the linear approximation is sufficient.

The intersection with delegation yields:

$$W^{**} = \frac{T_{overhead}}{\beta(1 + (P - 1) \cdot v) - \gamma} \quad (18)$$

At $P = 1$: $W^{**} = W^* = 37.5$ min. At $P = 5, v = 1$: $W^{**} \approx 7$ min. AI is optimal only for microtasks.

A Two-Dimensional Map: Size $\times$ Urgency

AI starts instantly (latency = 0). Delegation requires $T_{overhead}$. For urgent tasks, latency may matter more than T_{total} .

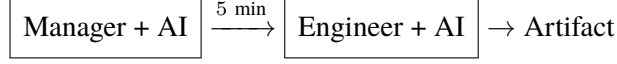
	Not urgent	Urgent
$< W^{**}$	AI	AI
$W^{**} - W^*$	Depends on P	AI (latency is critical)
$> W^*$	Delegation	Delegation, with a first pass by AI
$> 1 \text{ day}$	Delegation only	Decomposition $\rightarrow$ urgent tasks with AI

Quality shifts the threshold: high $Q \rightarrow$ high $I_Q \rightarrow W$ above the threshold $\rightarrow$ delegation. Prototypes go to AI; production goes to delegation.

6. The Fractal Chain

The Final Delegate Works with AI

In 2026, the engineer at the end of the chain also works with an AI agent:



The engineer uses AI more effectively than the manager: the engineer's C_{shared} with the codebase is richer (a better **model**), V_{in} is higher, and prompts are more precise. AI reduces $T_{engineer}$ (coefficient $\delta < 1$), while $T_{initiator}$ remains small. Delegation with AI is strictly better than delegation without AI; both are strictly better than direct work with AI when $W > W^*$.

Each Level Is a Pair with Its Own Window

$T_{overhead}$ differs by level: manager ~ 30 min, tech lead ~ 40 min, engineer ~ 25 min.

Level	P	$T_{overhead}$	W^{**}	Class of AI tasks	Output
Manager	5	~ 30 min	~ 7 min	Draft technical specification, solution structure	Information packet
Tech lead	3	~ 40 min	~ 15 min	Specifications, feature decomposition	Tasks
Engineer	1–2	~ 25 min	~ 31 min	Code, tests, refactoring	Artifact

One law governs all of this: $W^{**}(P) = T_{overhead}/(\beta(1 + (P-1)v) - \gamma)$. Manager: $30/4.4 \approx 7$. Tech lead: $40/2.6 \approx 15$. Engineer: $25/0.8 \approx 31$. The higher the level, the larger P , the narrower the window, and the more abstract the tasks. The strategy is not $N \rightarrow 1$, but $N = \text{const}$ with $\delta_k \downarrow$ at each level.

A Parallel Pain: The Engineer's Identity

For the manager, AI destroys parallelism. For the engineer, AI destroys something else—the **identity of the designer**. This is a parallel pain that deserves to be named explicitly.

Traditionally, the engineer designs from a blank page: reads the task, builds a model of the domain, chooses data structures, writes code. Each of these stages is an exercise in building one's own model. Working with AI inverts the regime: the engineer no longer designs from first principles; the engineer **verifies the guesses of a stochastic subordinate that never learns** (in the standard LLM configuration, there is no persistent memory of how this team solves this particular kind of task).

This is not merely cognitive load—we have already described its mechanics through Bainbridge and Zeigarnik. It is **erosion of competence**. The ability to build a model independently is like a muscle: it requires regular use. An engineer who spends years checking AI drafts instead of writing from scratch gradually loses the ability to design independently. A starting point is always proposed; the ability to find that starting point oneself atrophies. This is not a hypothesis from the future—it is a direct consequence of the fact that cognitive skills obey the use-it-or-lose-it principle (documented, among other places, in studies of navigational-skill degradation among GPS users).

In factorization terms, to be the **Model** (§7), the human must possess a full model of the domain—including the tacit layers $C_{implicit}-C_{nonverbal}$ —and those layers are built only through active design and through living with the consequences of one’s decisions. An engineer turned into a verifier loses access to the mechanism by which these layers are built. The paradox is that a system that requires the human to provide the “Model” component in order to work effectively systematically destroys the conditions under which that component is reproduced.

The practical conclusion for an organization is clear: **an engineer should not work with AI 100% of the time**. Some share of design from a blank page is not a luxury; it is hygiene for maintaining professional form. Just as an athlete cannot remain an athlete by only watching recordings of training sessions and commenting on them, an engineer cannot remain an engineer by only verifying LLM drafts.

Role Matrix

E = executes, A = assists, R = reviews.

Stage	Manager	Tech lead	Engineer	AI
Goal formulation	E	—	—	A
Decomposition	R	E	—	A
Specification	—	E	R	A
Implementation	—	—	E	A
Verification	—	R	E	A
Acceptance	E	R	—	—

Invariant of the current regime: only the human appears in the “E” column. Responsibility requires a model component (C_{shared} with the organization), and LLMs do not possess it. Whether this changes under a fundamentally different architecture of context transmission remains an open question.

7. The Pair as a Factorized Regulator

Definition

The node of the information graph is not a human, but a **pair** (Human + AI): $\text{Node}_k = (\text{Human}_k + \text{AI}_k)$. The pair is indivisible. The structure of the pair is a direct realization of the factorization theorem:

$$\text{Pair} = \text{Human (Model)} \times \text{AI (Filter)} \quad (19)$$

The human is the “Model” component: goal-setter and selector. The human sets the *why* and the *where to* (goal-setting is noncomputable—it requires a model of the world that does not fit inside a context window). The human evaluates AI output as a reward function: “is this what is needed?” The human filters through non-formalizable context (politics, relationships, intuition). The model of the system is the human with all five layers of C_{shared} .

AI is the “Filter” component: generator and verifier of completeness. It makes the implicit explicit (asks the receiver’s questions before transmission), structures the output (into a formalized packet), generates a checklist of omissions, and anticipates re-encoding into the language of the next level. The noise filter is AI insofar as it discards the irrelevant and leaves the signal.

When both components function, the pair is a minimal good regulator. When AI does not filter (no C_{shared} , no model of the receiver), the factorization collapses, and the human bears both roles.

Compression quality is the **product** of the two components:

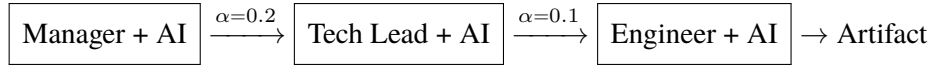
$$\text{Quality}_{pair} = f(\text{Model}_{human} \times \text{Filter}_{AI}) \quad (20)$$

Filter without model $\rightarrow$ noise. Model without filter $\rightarrow$ lossy signal. Model $\times$ Filter $\rightarrow$ compressed, lossless signal. $L_{pair}^{out} < L_{single}^{out}$. A cascade of three pairs with losses of 5%, 5%, and 15% yields $L_{total} = 23\%$. Three single agents with losses of 15%, 10%, and 30% yield $L_{total} = 46.5\%$. Twice as little loss along the chain.

Paths Between Pairs

Every transition is **re-encoding**: business goal $\neq$ feature $\neq$ specification $\neq$ code. AI participates both in the formation of the packet and in its unpacking.

Downward paths (task-setting): the receiver reconstructs what was omitted from C_{shared} .



$$I_{received} = I_{total} \cdot [\alpha + (1 - \alpha)(1 - L)] \quad (21)$$

L is the share of the non-explicit part that cannot be reconstructed (at $\alpha = 0.2$, $L = 0.15$: what is lost is $1 - 0.88 = 12\%$). The losses are L_{why} (the manager knows *why*, the tech lead receives only *what*) and $L_{alternatives}$ (the tech lead considered alternatives, the engineer receives only the chosen one). AI reduces L on both ends: on the sender’s side it explicates the implicit, and on the receiver’s side it detects omissions.

Upward paths (reporting): the consequences are the opposite. The manager cannot reconstruct technical detail from shared context—the manager’s C_{shared} with the code is sparse. What is not transmitted is lost. Let us use ρ (the retention coefficient):

$$I_{manager}^{received} = I_{engineer}^{sent} \cdot \rho^{eng \rightarrow lead} \cdot \rho^{lead \rightarrow manager} \quad (22)$$

Two transitions with $\rho = 0.2$: the manager sees 4% of the engineer’s information.

In factorization terms, on downward paths **the filter works**—the receiver (tech lead, engineer) possesses C_{shared} with the subject domain and reconstructs what was omitted. On upward paths, **the filter is absent**. The engineer transmits signal (status, blockers) mixed with two kinds of noise: excessive technical detail (complexity noise) or, conversely, excessive smoothing (political-correctness noise). The manager does not possess a model of the codebase sufficient to separate one from the

other. The tech lead filters partially, but even the tech lead’s model of “what matters to management” is incomplete.

Downward path: $\text{Regulator} = \text{Model}_{\text{receiver}} \times \text{Filter}_{\text{receiver}}$ —both are present, so factorization works. Upward path: $\text{Model}_{\text{receiver}} \approx 0$ for technical details—factorization collapses, because there is nothing with which to filter.

Three possible compensations (all of them hypotheses to be tested):

Bypass. The manager’s AI partner obtains direct access to lower-level artifacts (tickets, PRs, velocity, metrics) and builds a model that the manager does not possess. This restores the filtering component—but violates the hierarchical model and may undermine trust along the chain.

Structured protocol. The engineer’s AI partner forms a report from a template, separating facts (what was done, what blocks) from interpretations (risk assessment, schedule forecast). The template is a surrogate filter: it does not separate signal from noise, but it organizes information so that the receiver can filter faster.

Two-sided AI bridge. AI on the sender’s side and AI on the receiver’s side exchange machine-readable context ($C_{\text{shared}}^{AI \rightarrow AI}$), bypassing the text channel. This increases ρ —but is not yet implemented in standard tools.

The Structural Filter: Mapping onto Beer’s VSM

All three compensations share one principle: **the filter on the upward channel must be deterministic** (§2). This rules out purely statistical LLMs in the role of filter—their errors multiply along the chain and destroy the regulator’s minimality. The correct architecture is a structural filter grounded in a formal schema rather than probabilistic inference.

This construction maps naturally onto Stafford Beer’s Viable System Model (VSM):

VSM System	Function	Filter implementation
S1 (Operations)	Pair-nodes at the engineering level	Task execution; output = artifacts with formal invariants
S2 (Coordination)	Horizontal paths between pairs	$\alpha = 0.03$: common C_{shared} works without an additional filter
S3 (Optimization)	Internal coordination of resources	Joint planning; exchange of statuses between pairs on the same level
S3* (Audit)	Upward channel to tech lead/manager	Structured protocol : packet schema + aggregation by invariants
S4 (Development)	External context, strategy	Bypass to market metrics + structured analytic reports
S5 (Policy)	Goal-setting, identity	Human; C_{implicit} is non-transmissible

The key shift is this: in S3* the filter is not the LLM, but the **schema of the report packet** with explicit semantics (field types, invariants, references to state). The engineer’s LLM partner fills the schema but does not interpret its content. The manager’s LLM partner renders the content of the

packet into text but does not make decisions for the manager. Filtering occurs at the level of the schema—deterministically, verifiably, and with preservation of minimality.

The three hypotheses above are three mechanisms for building such a structural filter. They differ only in where the schema lives:

- **Bypass:** the schema lives in the source (ticket tracker, CI/CD, observability)—AI reads lower-level formalized artifacts directly.
- **Structured protocol:** the schema lives in the channel—a report packet with mandatory fields, filled jointly by a human and an LLM.
- **AI bridge:** the schema lives in the exchange between graphs—a machine-readable protocol, with the LLM present only at input and output to translate into human language.

All three are variants of a single pattern: **LLM as interface, deterministic graph as filter**. This restores factorization on upward paths.

Horizontal paths (coordination): maximal C_{shared} , $\alpha = 0.03$, minimal losses. AI on both sides accelerates review.

The Semantic Damper at A2A Interfaces

A separate problem arises on horizontal paths between graphs from different domains—when node A (development) exchanges state with node B (marketing, finance, compliance). The ontologies of these graphs do not coincide by construction: a “velocity decline” in development is not the same thing as a “release delay” in marketing, even though a correlation exists. Any attempt to hardcode an API contract between them leads to the classical microservice-architecture problem: the cost of maintaining the contracts exceeds the value of the services themselves, and any local increase in complexity breaks the integration.

The solution is to move stochasticity **to the boundary**, not inside the node. The synthetics (LLMs) at both ends of the A2A channel play the role of **semantic dampers**: graph A deterministically forms its formal packet, synthetic A translates it into a flexible linguistic format with $C_{explicit}$ for the adjacent domain, synthetic B resolves the ontological conflict and deterministically fits the result into the invariants of graph B. Each graph remains the minimal regulator of its own domain; stochasticity is localized in the communication channel.

But the channel itself then becomes a source of a specific threat—**type II errors** (§2). LLMs were trained on corpora in which textual smoothness correlates with authorial competence; RLHF reward models rewarded confident tone. As a result, modern models systematically smooth signals of uncertainty: “critical risk” turns into “considerations requiring attention,” “we do not know the date” into “expected in Q2.” A type I error is visible as information loss; a type II error is invisible as distortion into a smooth falsehood. The recipient’s Zeigarnik loop closes falsely: the person believes they received a clear signal, when in fact they received polished noise.

Protection against type II errors at A2A interfaces is built on three levels:

1. Semantic discretization at output. An ordinary REST contract checks syntax (field `deadline` = ISO-8601 `datetime`); that is sufficient for type I, but not for type II. Semantic invariants require

discretizing key metrics into an ordinal scale of 3–5 levels: `velocity_trend` $\in$ {up, flat, down}, `risk_level` $\in$ {green, amber, red}, with a mandatory field `uncertainty_source` for all non-green states. Discretization forces the LLM to **choose** one label from a limited set instead of describing nuances in prose. “Amber with `uncertainty_source=technical_debt`” is substantially harder to fake with smooth text than a free paragraph saying “there are considerations.”

2. Circular verification via ontology mapping. LLM translation is asymmetric: the direction “formal schema $\rightarrow$ natural language” is stable, the reverse direction unstable. This gives an engineering lever. If synthetic A translated packet A into text, and synthetic B translated the text into packet B, then the results should be comparable after applying an a priori ontology map $\phi : A \rightarrow B$. If $\|\text{Packet}_B^{\text{claimed}} - \phi(\text{Packet}_A)\| > \theta$, semantic drift has occurred: the packet is rejected or escalated to a human. This is a **deterministic check of a stochastic translation**; most boundaries self-check, and the human is involved only when divergence appears.

3. Adversarial proofing. A third LLM in the role of red team receives the task: “find signs in the packet that the real situation is worse than the description.” Markers include removed hedge words, excessively precise quantification of uncertainty, and a green status combined with a non-null uncertainty source. This is not complete protection (the red team is itself an LLM), but it shifts the probability: single-layer smoothing passes, whereas two-layer smoothing (smooth text plus successful adversarial proofing) requires an order of magnitude more consistency, which stochastic systems do not possess without collusion.

Availability of ontology mapping ϕ . The second mechanism relies on an a priori map $\phi : A \rightarrow B$ between the schemas of the two graphs. This map is not a free resource, but an **architectural artifact** that requires investment. Three regimes of availability are possible:

- **ϕ is stable:** the domains share an upstream taxonomy. “Deal value” in CRM maps unambiguously to “revenue” in the financial graph—the same metric under different names. This occurs in mature organizations with data governance or in systems from a single vendor. Circular verification then works without qualification.
- **ϕ is partial:** some fields map cleanly, others do not. A typical case: `risk_level` has a shared ontology, while `uncertainty_source` differs across domains. Verification applies to the mappable fields; the rest go through discretization and adversarial proofing.
- **ϕ does not exist stably:** the ontologies of A and B are incommensurable (for example, “technical complexity” in development and “operational risk” in compliance are different category spaces with no regular mapping). In this case circular verification is unavailable; protection reduces to two mechanisms (discretization + adversarial proofing), which lowers robustness against type II errors but does not eliminate it.

Constructing ϕ is a separate project task requiring weeks or months of semantic modeling work between two mature graphs. Possible approaches include manual definition by data architects, learning from pairs of aligned packets, and iterative refinement with a human in the loop under escalation.

The important conclusion is that **the capacity of a pair of graphs for A2A communication depends on the maturity of the ontological work between them**. If ϕ does not exist and is expensive to build, that is a signal that the boundary between the graphs was drawn along organizational rather than semantic seams, and ought to be reconsidered.

Why all three require deterministic graphs. Semantic discretization is meaningless if the receiving graph is a free-form report (there is nothing to choose from). Circular verification is impossible without ϕ between formal schemas. The adversarial reader has no anchor points without the invariants of graph B. This closes the objection “then let us just hardcode the API”: graph schemas **do not coincide by construction**, and harmonizing them means years of work by a standardization committee. The LLM bridge allows each graph to remain optimal in its own domain while delegating ontological translation to a stochastic layer—but that layer is isolated and checkable.

The honest remainder. Even with all three mechanisms, the adversarial reader is still an LLM, ontology mapping may be underdetermined, and semantic discretization requires preliminary ontological work. Type II error is not eliminated—it is **substantially reduced** and made visible where it was previously invisible. This is hygiene, not certification. But within the factorization of §2, hygiene is enough: the boundary carries invariants, invariants cut away a sufficient share of type II errors for the receiving graph to preserve minimality with controlled ε . Rigor returns not to the center of the system (that is where the LLM lives), but to its **boundaries** (that is where graph invariants live).

Three properties of the topology follow: homogeneity of nodes (all are pairs, differing only in parameters), asymmetry of paths (downward paths unpack, upward paths compress), and AI everywhere but in different roles (formulation at the top, generation at the bottom, translation at the interfaces).

8. Meta-Example: This Essay

This essay was written in a pair (Author + AI) and is a live demonstration of the model described. A fair evaluation requires acknowledging the contributions of both participants—not only the pathologies, but also what works.

The Meta-Example Paradox

This essay is a **counterexample to its own thesis in the strong form**. In §1 I claimed that “the total time for a task of a given quality has not decreased.” But the text you are reading would not have been written in a single day without AI—formulas, tables, verification, literature search, and 400+ lines of formal exposition would have taken **weeks**. AI radically reduced T_{author} .

How does this fit the thesis? The contradiction is resolved through the essay’s internal model. I_Q for formalization work (drafting, article writing, synthesis) is substantially lower than for production work: it requires mainly $C_{explicit}$ (concepts, formulas, sources), which AI sees almost completely. $C_{implicit}$ is needed to correct direction (which the author did in ~ 30 messages), but not for the full volume of the text. This explains why the pair worked with $\beta \approx 0.9$ by volume, yet efficiently in time.

Hence a **narrowing of the central thesis**. Not “AI does not reduce $T_{initiator}$ at all,” but rather: **for high-context production work (production code, architecture on which money depends), AI**

often does not reduce the initiator's time as much as the industry promises. For formalization work (synthesis of ideas, documentation, drafts), it reduces time radically, because $C_{implicit}$ is needed only at critical points, not throughout the full volume. The boundary between the two regimes is the share of I_Q that requires nonverbalizable context.

This is not a weakening of the thesis but its refinement. The AI productivity paradox is real—but for a specific class of work. The meta-example shows both classes: the author works in a production regime on books, arXiv papers, and product architecture (Corezoid and Simulator are platforms for orchestrating business processes on the basis of actor graphs, developed by the author since 2009)—and there the essay's model applies. The essay itself, by contrast, is a formalization artifact and an example of the opposite class.

What AI Did

AI produced the main volume of the artifact. Formalization: $I_Q = \text{const}$, $V_{transfer} = \min(V_{out}, V_{in})$, $W^* = T_{overhead}/(\beta - \gamma)$, $I_{received} = I_{total}[\alpha + (1 - \alpha)(1 - L)]$, and the entire surrounding arithmetic and verification. Structure: it proposed a 12-section architecture that, through iterations, became a 9-section one. Literature: it found, cited, and connected 16 sources—from Shannon to Dell'Acqua. Tables: 10+ comparative matrices. Verification: it detected the error $W'_Q = \alpha W_Q$ (it should have been α^2 —the formula was removed), the double semantics of α , the discrepancy in the tech lead's W^{**} , and the inconsistency of units ($250\times \rightarrow 200\times$). Text: ~ 400 lines of coherent formal essay in Russian in a single session.

What the Author Did

The author produced ~ 30 messages, each between 3 and 30 words long. In total, ~ 500 words across the whole session. By volume, less than 1% of the text. But every message changed the direction:

- “We need to introduce the metrics V_{out} and V_{in} ” $\rightarrow$ which led to the law of the bottleneck (§2).
- “Add Miller's number” $\rightarrow$ the bottleneck received neurophysiological grounding.
- “Show that the final delegate works with AI” $\rightarrow$ the three-link chain appeared.
- “Not like that! Everyone works in pairs” $\rightarrow$ the topology was rewritten: node = pair, not human.
- “The human is not a passive participant either” $\rightarrow$ the pair became symmetrical.
- “Where is my theorem of noise suppression?” $\rightarrow \dots \rightarrow$ “The point is not citations but analogies” $\rightarrow$ factorization became the skeleton.

Without the author, AI would have produced a formally correct but conceptually flat text—without factorization as the generative principle, without the five functions as noise filters, and without the pair as an indivisible node.

Factorization in Action

Component	Author	AI
Model (direction, judgment)	Theorem as lens, key turns, elimination of false paths	Structural proposals, alternative phrasings
Filter (quality control)	Selection of generative analogies from decorative ones	Arithmetic verification, logical checks
Generation (volume)	~500 words in the session	~400 lines of text, 16 references, 10+ tables
By volume	~1%	~99%
By direction	~90%	~10%

Both contributions were necessary. The quality of the pair is the product of the components: $f(\text{Model} \times \text{Filter})$. Zero out either one, and the result collapses. Without the author, AI would have produced 400 lines of noise ($\eta \ll 1$). Without AI, the author would have produced 10 lines on a whiteboard—deep, but not formalized.

Chronology: How Factorization Was Not Seen

From the very first message, the author knew that the theorem of factorization would explain the structure of the work. AI did not know this—and passed through four stages:

1. **Ignoring.** AI built the model from scratch without connecting it to the theorem of factorization. The theorem did not occur to it because for AI it was text, not a lens.
2. **Citation.** “Where is my theorem?”—AI inserted “(Vityaz, 2026a).” A bibliographic decoration.
3. **Decorative analogy.** “The point is not citations”—AI walked through the sections and attached labels: “§4—in the terms of factorization.” Post hoc, not a priori.
4. **Generative principle.** Only after an explicit instruction did AI rewrite the essay with factorization as its skeleton.

Four stages—four interventions by the author. C_{shared} of AI with the theorem of factorization = the text in the context window. The five functions of context (§4) for this theory are:

Function	AI	Author
Prediction	0 — did not predict that the theorem would explain the pair	1 — knew it from the first message
Intuition	0 — no “night in production” with factorization	1 — years of applying it in Simulator
Networked context	0 — does not know the history of application	1 — built products on the theorem
Real time	0 — sees only text	1 — sees progress and corrects
Filtering	0 — does not distinguish a generative analogy from a decorative one	1 — filters instantly

Five zeros for AI, five ones for the author—for this particular theory. For other tasks (arithmetic, literature search, table generation), the picture reverses: AI outperforms. The pair works not because one is better than the other, but because their competencies are **orthogonal**: the human is strong where AI is weak, and vice versa. Factorization is not a hierarchy, but a division of labor between complementary components.

9. Conclusion

Work is information processing. The volume I_Q is determined by the task, not by the tool—the law of conservation. Processing speed is limited by Miller’s number—the law of the bottleneck. The minimal good regulator factorizes into a model and a filter—the law of factorization.

Human context is a five-layer codebook ($C_{explicit}$, $C_{implicit}$, $C_{ambient}$, C_{social} , $C_{nonverbal}$), of which AI sees only one layer. Each of the five mechanisms of context is a noise filter strengthened by the depth of lived experience. Depth is a qualitative transition from reference material to intuition.

In the pair (Human + AI), the human is the model (judgment, context, goal-setting), while AI ought to be the filter (structure, completeness, suppression of noise). When AI does not filter, factorization collapses, the human bears both roles, and $\beta \rightarrow 1$. This explains the AI productivity paradox, Dell’Acqua’s “jagged frontier,” and the Jevons paradox.

The key orders of magnitude in the model are these: delegation threshold $W^* \sim 30\text{--}50$ min (without opportunity costs) and $W^{**} \sim 5\text{--}10$ min (when $P \sim 5$). Cascade of losses: three pairs lose roughly half as much information along the chain as three single agents. Upward paths: with typical parameters $\rho = 0.2$, the manager receives about **4% of the engineer’s information** across two hierarchical levels— $\rho^2 = 0.04$. The reader obtains concrete numbers for their own team by substituting their own parameters into the formulas of §§5–7.

The optimal configuration is a fractal chain of pairs in which each level uses AI within its own window $W^{**}(P)$. The manager uses AI for seven-minute microtasks. The tech lead uses it for fifteen-minute specifications. The engineer uses it for thirty-minute blocks of code. The number of participants does not decrease. Everyone accelerates. The informational chain is preserved. Compression works.

The true metric of AI productivity is not generation speed. It is the compression coefficient $\alpha = I_{explicit}/I_{total}$. As long as $\alpha \approx 1$ in both directions, we are not working with AI. We are working instead of it. To restore factorization means giving AI a model (C_{shared} , all five layers) and the ability to return a diff rather than a dump. In the language of the automotive industry, we are at Level 2—hands on the wheel, eyes on the road. The goal is not Level 5 (which is not visible on the horizon under the current architecture of context transmission), but Level 4: AI that filters well enough for the human to take their hands off the wheel and move to strategy.

The three hypotheses from §7—**bypass** (direct AI access to lower-level artifacts), **structured protocol** (AI forms reports from templates, separating facts from interpretations), and **two-sided AI bridge** ($C_{shared}^{AI \rightarrow AI}$ between participants)—are concrete technical paths from Level 2 to Level 4. Bypass expands AI’s C_{shared} via access to data. Protocol gives AI a ready-made model of filtering. The bridge removes the human cognitive boundary at the interfaces between pairs. Each path is a separate engineering problem, testable by the metric α : if after implementation the compression coefficient in

the Human $\leftrightarrow$ AI pair falls by an order of magnitude, the path works.

AIomatics: Three-Component Factorization

Before introducing the discipline, one internal tension must be resolved. In §7, I called AI the “Filter” component of the pair. In §2, I stated that a statistical filter violates minimality. These claims are compatible if “AI” in §7 is understood not as a bare LLM, but as a **hybrid subsystem Graph + LLM**: a deterministic actor graph plays the role of filter, while the LLM plays the role of interface (translation between natural language and formal schema). When, today in 2026, the pair Human + bare LLM performs badly, this is precisely the symptom of the graph’s absence. The normative model of §7 presupposes the restoration of the graph. The discipline that studies and engineers this restoration is AIomatics.

AIomatics (AI + -omatics, by analogy with *informatics*, *cybernetics*, *mathematics*) is the discipline that studies AI systems as regulators and designs their architecture from the requirement of minimality.

Subject: composite systems Human $\times$ Graph $\times$ LLM, considered as good regulators (in the Conant–Ashby sense), with an explicit separation of Model and Filter.

Method: factorization analysis. Every architecture is checked for (a) completeness of the Model (does it cover the variety of the task?), (b) determinism of the Filter (does it satisfy the condition of minimality from §2?), and (c) cascade losses (is factorization preserved along the chain of VSM levels?).

Design principles:

- A stochastic component cannot be the Filter—only the Interface or the Generator.
- A deterministic schema (an actor graph with invariants) is the carrier of the Filter.
- The human is the carrier of the full Model (with $C_{implicit}$, lived experience, judgment).
- Minimality of the regulator is checked formally, not by tests.

Positioning relative to neighboring disciplines:

Discipline	System center	Filter	Limitation
Classical ML / LLM	Neural network	Statistical	Violates minimality
Neurosymbolic AI	Neural network + symbolic blocks	Mixed	Filter is partially stochastic
Formal methods	Logic / model checking	Deterministic	No AI interface
AIomatics	Actor graph	Deterministic	LLM is peripheral

In focus: architectural patterns for restoring factorization, metrics of regulator minimality, cascade analysis of losses across VSM levels, and the design of paths between pairs.

Out of focus: the internal structure of LLMs, optimization of statistical models, and training on data. Those tasks are handled by classical ML; AIomatics takes their results as given and designs the composition.

Two Classes of Architectures

All three hypotheses from §7 work only if the condition of filter determinism from §2 is satisfied. This distinguishes two classes of architectures:

Statistical (the current state). The filter is the LLM. Error in a fraction ε of cases multiplies across the chain of pairs: $L_{total} = 1 - \prod_k (1 - L_k)$. For illustration, let us take $L_{LLM} \approx 0.15$ as an order-of-magnitude estimate for medium-complexity tasks (hallucinations + inaccuracies; real values range from $\sim 5\%$ on refactoring to $30\%+$ on integration with an unfamiliar codebase). Across three levels, the cascade yields losses of approximately 39% —worse than single agents without AI. The exact number matters less than the qualitative result: this architecture cannot reach Level 4 in principle, because every additional hierarchical level increases cumulative error.

Hybrid (AIomatics). The filter is a deterministic actor graph with formal semantics. The LLM is the interface: it parses human intent into a formal packet on input and translates graph output into human text on output. Factorization becomes three-component:

$$\text{Regulator}_{min} = \text{Human}(\text{Model}_{full}) \times \text{Graph}(\text{Model}_{domain} + \text{Filter}) \times \text{LLM}(\text{Interface}) \quad (23)$$

Distribution of roles:

Component	In the factorization	Nature	Function
Human	Model_{full}	Lived, with $C_{implicit}$	Goal-setting, exceptional cases, graph updates
Actor graph	$\text{Model}_{domain} + \text{Filter}$	Deterministic, symbolic	Invariants, preconditions, routing, state preservation
LLM	Interface	Statistical	Translation between natural language and formal protocol

The key point is this: **filtering is not in the LLM, but in the graph.** The LLM remains a generator and translator—roles compatible with statistical nature. The graph guarantees regulator minimality through deterministic behavior and formally verifiable invariants.

Architectural pattern as a case of practice preceding formalization. The author has been using a deterministic orchestrator with peripheral stochastic components in production systems since 2009 (the Corezoid and Simulator platforms), long before the formalization of the Vityaz–Ashby theorem. This is a case in which engineering intuition outran theory by more than a decade and a half: the theorem explained post factum why the pattern worked, but did not generate it. This matters for the epistemology of the essay: the central thesis is not a speculative frame, but a retrospective formalization of a pattern that has undergone industrial validation. Alternative implementations of the same principle (constrained LLM decoding with formal grammars, neurosymbolic architectures, verified reward models) share the same core: the stochastic component must not occupy the role of filter. Actor Graph Theory is one path, not the only one.

The honest trade-off. The requirement of a deterministic filter is not free. It shifts system complexity from training the stochastic model (where it is amortized in gradient descent) to **designing the graph** (where it is paid for through human semantic modeling—see the discussion of the availability of ϕ in §7). In domains with rich formal structure (finance, compliance, code), this cost is justified—the graph serves for decades. In domains with diffuse ontology (creative tasks, research dialogue), graph design may cost more than the acceptable level of LLM noise; there the statistical architecture may be empirically more effective, even while sacrificing minimality. The Vityaz–Ashby theorem describes the **upper bound of regulator quality**; statistical architectures may be economically optimal below that bound for specific tasks. The disciplinary task is to know **where in the architectural space** your task lies.

Work is information processing. Its volume cannot be minimized: I_Q is invariant. Its processing cannot be accelerated through pure generation: the bottleneck is human cognitive throughput. The only path to the minimal regulator is to **factorize it so that the filter is deterministic**. At present, we are at Level 2: LLM as filter, cascade losses, monitoring fatigue. Almatix is the path to Level 4: LLM as interface, graph as filter, minimal regulator, restored factorization. For tasks in which minimality is critical, this is a normative requirement. For tasks in which compromise is acceptable, the framework provides an instrument for estimating how much minimality is lost.

Genre and Limitations

The text is written in the genre of an **essay with theorems** rather than a peer-reviewed article. This implies three concrete consequences for the attentive reader.

First, the mathematical apparatus (the determinism inequality in §2, the threshold formulas in §5, and the factorization in §7) functions as an **analytic scaffold**, not as a proof basis in the strict academic sense. The multiplicative version of the type I / type II inequality, the empirical calibration of μ and ρ , and the formal verification of the Vityaz–Ashby theorem in categorical form are all tasks for **separate mathematical works** toward which the text points but which it does not itself solve.

Second, the numerical illustrations ($W^* \approx 37.5$ min, $\rho^2 \approx 4\%$, cascade losses of 23% vs. 46.5%) are the result of plugging expert estimates of parameters into derived formulas, not empirically measured magnitudes. They demonstrate a method; they do not confirm the hypothesis. Empirical validation is the task of experimental studies in the style of Peng et al. or Dell’Acqua et al., ideally conducted independently of the author of the theoretical model.

Third, the key observations about managerial pain, the erosion of engineer identity, and the accumulation of management debt are based on many years of observation in production systems (2009–2026), but have not been formalized as statistically significant empirical findings. They function as **testable predictions** of the model, awaiting validation through RCTs or longitudinal studies.

The proper use of the text is as a **framework for generating hypotheses** and a **language for discussing the phenomenon**, not as a final answer to the question “how does AI change work?” Final answers require empirical work, formal mathematics, and case studies—three classes of work absent here by genre choice.

April 2026

References

1. Jevons, W. S. (1865). *The Coal Question*. Macmillan.
2. Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.
3. Wiener, N. (1948). *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press.
4. Ashby, W. R. (1956). *An Introduction to Cybernetics*. Chapman & Hall.
5. Miller, G. A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2), 81–97.
6. Polanyi, M. (1966). *The Tacit Dimension*. University of Chicago Press.
7. Conant, R. C., & Ashby, W. R. (1970). Every good regulator of a system must be a model of that system. *International Journal of Systems Science*, 1(2), 89–97.
8. Brooks, F. P. (1975). *The Mythical Man-Month*. Addison-Wesley.
9. Bainbridge, L. (1983). Ironies of automation. *Automatica*, 19(6), 775–779. [https://doi.org/10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8)
10. Sweller, J. (1988). Cognitive load during problem solving. *Cognitive Science*, 12(2), 257–285.
11. Nonaka, I., & Takeuchi, H. (1995). *The Knowledge-Creating Company*. Oxford University Press.
12. Cowan, N. (2001). The magical number 4 in short-term memory. *Behavioral and Brain Sciences*, 24(1), 87–114.
13. Mark, G., Gonzalez, V. M., & Harris, J. (2005). No task left behind? Examining the nature of fragmented work. *Proceedings of CHI 2005*, 321–330.
14. Mäntylä, T., & Sgaramella, T. (1997). Interrupting intentions: Zeigarnik-like effects in prospective memory. *Psychological Research*, 60(3), 192–199.
15. Koshino, H., Minamoto, T., Ikeda, T., et al. (2014). Anterior medial prefrontal cortex exhibits activation during task preparation but deactivation during task execution. *PLoS ONE*, 9(5), e97508.
16. Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
17. Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv:2302.06590*.
18. Dell’Acqua, F., McFowland, E., Mollick, E., et al. (2023). Navigating the jagged technological frontier. *Organization Science* (forthcoming).

19. Peng, S., Demirer, M., Cui, K. Z., et al. (2024). The effects of generative AI on high-skilled work. MIT Working Paper.
20. Vityaz, A. (2026a). On the necessity of noise suppression for minimal good regulators: Factorization theorems and a closure conjecture. ResearchGate. [link](#)
21. Vityaz, A. (2026b). Management Debt — Part I: Concept, metrics, and principles for attributing materialised debts to actor accounts. ResearchGate. [link](#)
22. Zeigarnik, B. (1927). Über das Behalten von erledigten und unerledigten Handlungen. *Psychologische Forschung*, 9, 1–85.
23. [NotebookLM](#)